Regularized Regression

Ridge regression, the lasso, and tuning with cross-validation

Statistical Computing for Data Analysis

Iowa State University

From cross-validation to regularization

Last time, we focused on estimating prediction error more realistically.

The main message was:

- training MSE is often too optimistic,
- test MSE is what we really care about for prediction,
- and cross-validation gives a practical way to estimate prediction error.

Today we use that idea for **model tuning**: we will choose a tuning parameter using CV.

Why do we need regularization?

Ordinary least squares works well in many settings, but it can be unstable for prediction when:

- we have many predictors,
- predictors are strongly correlated,
- or the fitted model is too sensitive to small changes in the data.

In those settings, least squares may fit the training data very well but generalize less well to new observations.

Regularization addresses this by adding a **penalty** that shrinks the coefficient estimates.

Ridge regression

Recall that least squares minimizes

$$\mathrm{RSS} = \sum_{i=1}^{n} \left(y_i - \beta_0 - \sum_{j=1}^{p} \beta_j x_{ij} \right)^2 .$$

Ridge regression minimizes the penalized objective

$$\mathrm{RSS} + \lambda \sum_{j=1}^{p} \beta_j^2,$$

where $\lambda \geq 0$ is a tuning parameter.

Interpretation:

- $\lambda = 0$ gives ordinary least squares,
- larger λ gives more shrinkage,

What does ridge regression do?

Ridge regression trades off two goals:

- fit the training data well, and
- keep the slope coefficients from becoming too large.

As λ increases:

- the coefficients are pulled toward 0,
- the model becomes more stable,
- variance tends to decrease,
- bias tends to increase.

This is another example of the **bias–variance tradeoff**.

Why do we standardize predictors?

The ridge penalty depends on the size of the coefficients.

If one predictor is measured on a much larger scale than another, then the penalty will not treat them fairly.

So before fitting ridge or lasso, we usually standardize the predictors.

$$\tilde{x}_{ij} = \frac{x_{ij} - \bar{x}_j}{s_j}.$$

Practical note:

- `glmnet()` standardizes predictors by default,
- but conceptually it is important to understand *why* we do this.

The lasso

The lasso uses a different penalty:

$$\mathrm{RSS} + \lambda \sum_{j=1}^{p} |\beta_j|.$$

Like ridge regression, the lasso shrinks coefficients toward 0.

But unlike ridge regression, the lasso can set some coefficients **exactly equal to 0**.

This means the lasso can do two things at once:

- regularization for prediction,
- variable selection for a simpler model.

Ridge versus lasso

Feature	**Ridge**	**Lasso**
Penalty	$\sum \beta_j^2$	$\sum \lvert\beta_j\rvert$
Effect on coefficients	Shrinks toward 0	Shrinks toward 0
Exact zeros?	No	Often yes
Main strength	Stability	Sparsity / variable selection

Prediction rule of thumb:

- if many predictors carry signal, ridge often does well;
- if only a few predictors matter, lasso often does well.

How do we choose λ?

This is where cross-validation enters.

For a grid of candidate λ values:

1. fit the model on the training folds,
2. compute prediction error on the validation fold,
3. average across folds.

Then we choose the value of λ with the smallest CV error.

After that, we refit the model using the selected λ and evaluate on the test set.

Example setup: the Credit data

We will use the `Credit` data from `ISLR2`.

Response:

- `Balance`

Predictors include both quantitative and qualitative variables, so we use `model.matrix()` to build the design matrix.

```
library(glmnet)
library(ISLR2)

data(Credit)

X <- model.matrix(Balance ~ ., data = Credit)[, -1]
y <- Credit$Balance

set.seed(1)
n <- nrow(X)
```

Ridge regression in R

We fit ridge regression with `alpha = 0`.

```
ridge_fit <- glmnet(X[train_id, ], y[train_id], alpha = 0)
plot(ridge_fit, xvar = "lambda", label = TRUE)
```

To choose the tuning parameter by cross-validation:

```
set.seed(1)
ridge_cv <- cv.glmnet(X[train_id, ], y[train_id], alpha = 0)

plot(ridge_cv)

ridge_lambda_min <- ridge_cv$lambda.min
ridge_lambda_1se <- ridge_cv$lambda.1se
```

Lasso in R

We fit the lasso with `alpha = 1`.

```
lasso_fit <- glmnet(X[train_id, ], y[train_id], alpha = 1)
plot(lasso_fit, xvar = "lambda", label = TRUE)
```

Cross-validation works the same way:

```
set.seed(1)
lasso_cv <- cv.glmnet(X[train_id, ], y[train_id], alpha = 1)

plot(lasso_cv)

lasso_lambda_min <- lasso_cv$lambda.min
lasso_lambda_1se <- lasso_cv$lambda.1se
```

Looking at the selected lasso coefficients

Because the lasso can set coefficients exactly to 0, it is useful to inspect the fitted coefficients at the selected value of λ.

```
coef(lasso_cv, s = "lambda.min")
coef(lasso_cv, s = "lambda.1se")
```

Usually:

- `lambda.min` gives the model with the smallest CV error,
- `lambda.1se` often gives a simpler model.

Comparing test MSE

We can compare ordinary least squares, ridge, and lasso on the held-out test set.

```
ols_fit <- lm(Balance ~ ., data = Credit, subset = train_id)

ols_pred <- predict(ols_fit, newdata = Credit[test_id, ])
ols_mse <- mean((y[test_id] - ols_pred)^2)

ridge_pred <- predict(ridge_cv, newx = X[test_id, ],
                      s = "lambda.min")
ridge_mse <- mean((y[test_id] - ridge_pred)^2)

lasso_pred <- predict(lasso_cv, newx = X[test_id, ],
                      s = "lambda.min")
lasso_mse <- mean((y[test_id] - lasso_pred)^2)

c(OLS = ols_mse,
  Ridge = ridge_mse,
```

What should we remember?

- Ridge and lasso are regularized versions of linear regression.
- Both introduce a tuning parameter λ.
- Ridge shrinks coefficients; lasso can also set some coefficients to 0.
- Predictors should be standardized before applying the penalty.
- Cross-validation is a practical way to choose λ.
- Final model comparison should be based on test-set performance.

Cross-validation is not just for assessment; it is also a tool for tuning models.